

Joint Embedding Predictive Architectures (JEPA) for Vision, Video, and Vision–Language

Modern AI often learns by **generating tokens** (words or pixels) sequentially, but this is wasteful for vision-language tasks. JOINT EMBEDDING PREDICTIVE ARCHITECTURE (JEPA) sidesteps this by predicting **continuous semantic embeddings** instead of discrete tokens[1][2]. In a JEPA model, a **frozen visual encoder** (trained with self-supervision) produces embeddings of the input image or video, and a *predictor* (a small Transformer) maps these embeddings (plus any text query) directly to the **semantic embedding** of the answer. A lightweight text decoder is only used *afterwards* to render human-readable text when needed[3][4].

This design delivers major gains: VL-JEPA (a vision–language JEPA) uses **50% fewer parameters** than comparable vision-language models and learns **~2× faster**, while achieving equal or better accuracy[5][6]. At inference, it can **skip most text decoding** by monitoring the semantic stream and only decoding when the meaning changes[7][8]. Below we explain how JEPA works for images, video, and vision–language, using diagrams and examples.

The JEPA Concept: Predict in Embedding Space

Conventional generative models (e.g. CLIP+LLM, Flamingo) connect vision and language by **token-based** decoding. They generate text token-by-token, needing to commit to specific words (e.g. “cat” vs “kitten”) and sequences. This forces the model to fit many distinct surface forms of the same underlying meaning. JEPA avoids this by operating in **dense semantic space**: it predicts a point in an embedding space where synonyms and paraphrases naturally cluster together[9][1]. As one blog puts it, “JEPA allows the system to handle uncertainty and ignore irrelevant details while maintaining essential information”[10].

In practice, a JEPA model has two encoders and one predictor (see figure below). First, a **Vision Encoder** (often a frozen, self-supervised ViT like V-JEPA 2) turns the input image or video into visual embeddings S_V [8†]. Second, a **Text Encoder** (Y-Encoder) maps the ground-truth text answer Y into a *target embedding* S_Y in the same space[8†]. Finally, a **Predictor Transformer** takes the visual embeddings (and any text query) and outputs a **predicted embedding** $\hat{S}_Y$ [8†]. During training, $\hat{S}_Y$ is pulled towards S_Y using a contrastive (InfoNCE) loss, encouraging $\hat{S}_Y$ to lie in the same semantic region as the true answer[3][4].

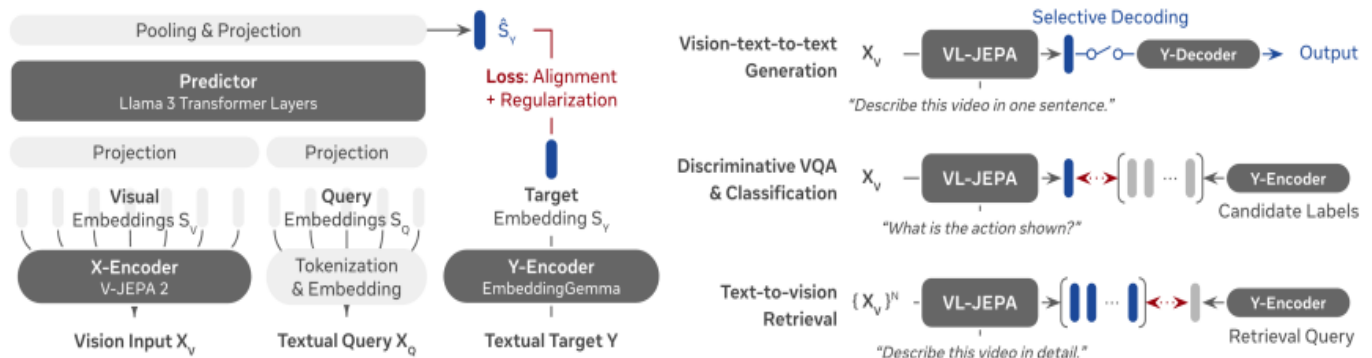


Figure: VL-JEPA (vision-language JEPA) architecture. A frozen video encoder (V-JEPA 2) produces visual embeddings S_v from images/videos. A Transformer predictor (based on Llama) takes S_v plus any text query embeddings S_q and outputs a predicted semantic embedding $\hat{S}_v$. A separate text encoder (Y-Encoder) maps the true answer into embedding S_y for the loss. At inference, a lightweight decoder converts $\hat{S}_v$ into output text only when needed.

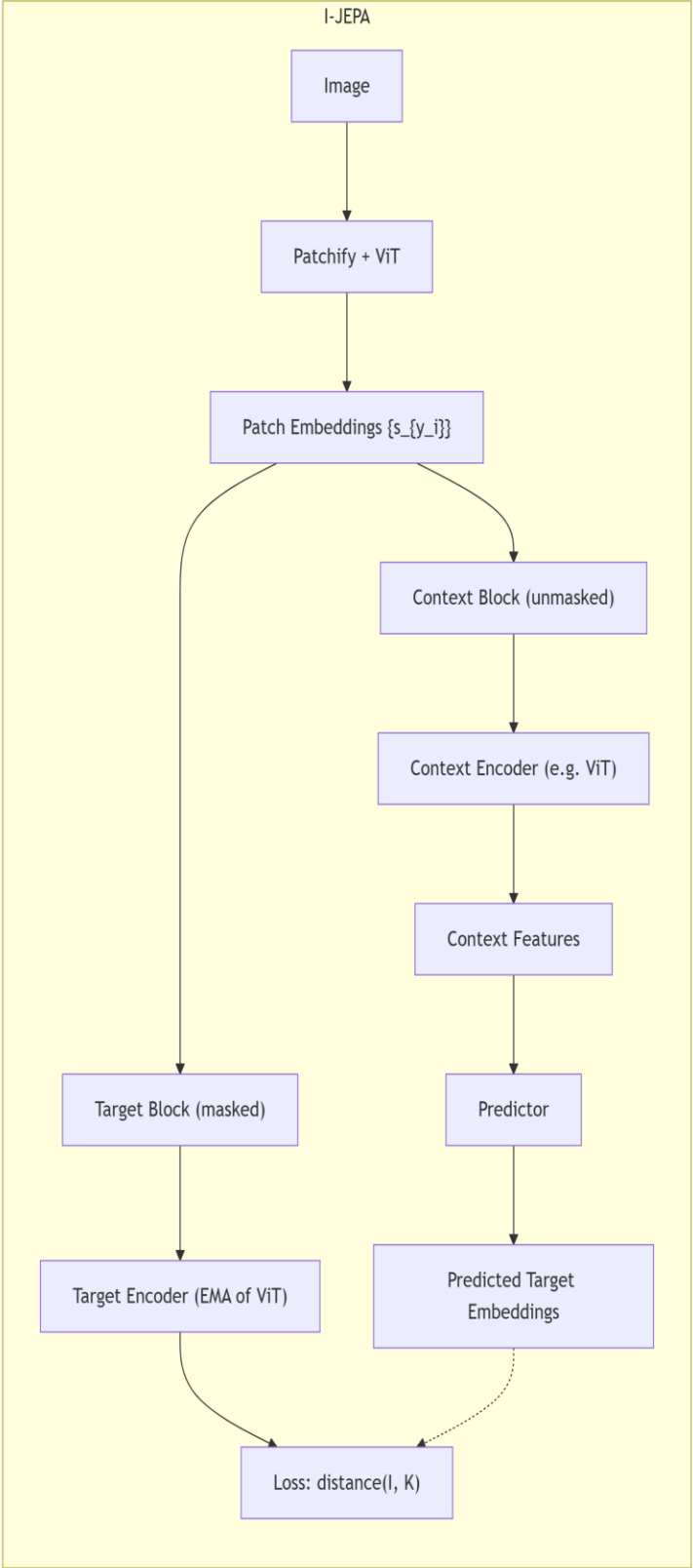
Why Predict Embeddings Beats Generating Tokens

- **Unimodal vs Multimodal Targets:** Consider a question "What color is the car?" with answers like "red" or "crimson." In token space these are entirely different sequences. In embedding space, their vectors are nearby. JEPA forces all valid answers to share one target cluster, simplifying learning[10][1].
- **Efficiency:** Predicting one embedding requires one forward pass (and no softmax over large vocab), versus sequential decoding. VL-JEPA uses a 490M-parameter predictor instead of a 1B+ auto-regressive decoder[5][6]. It converges roughly 2× faster on tasks like captioning and classification[6][11].
- **Handling Uncertainty:** Because embeddings can represent a distribution (a region), the model needn't commit prematurely to exact wording. It can hedge by aiming for the "right area" in semantic space[12].

In short, JEPA reframes vision-language tasks as **regression in semantic space** rather than language generation. It aligns with Yann LeCun's vision of AI learning a *world model* directly from sensory data[13][14].

I-JEPA: JEPA for Images

The simplest JEPA instance is **I-JEPA** (Image JEPA) for static images[15][9]. Here the task is self-supervised: from one part of the image (a *context* block), predict the representations of other parts (the *target* blocks). Concretely, an input image is split into many patches and passed through a ViT encoder to get patch embeddings. Then a random *context patch* (or block of patches) is chosen, and a larger overlapping *target block* is masked out. A predictor network sees only the context (with the target region masked out) and must predict the embeddings of the target block[15][16].



In words: the **context encoder** processes visible parts, a predictor guesses the hidden parts' embeddings, and a **target encoder** (an EMA copy of the same ViT) provides the true embeddings. The loss (often L1 or cosine) pulls the predictor's output towards the target encoder's output[17][15]. By masking *large* target blocks (not just small patches), I-JEPA forces the model to use high-level scene understanding. Empirically this learns very strong features quickly: one paper reports training a ViT-Huge/14 on ImageNet with I-JEPA in ~72 hours to reach high downstream accuracy[15][18].

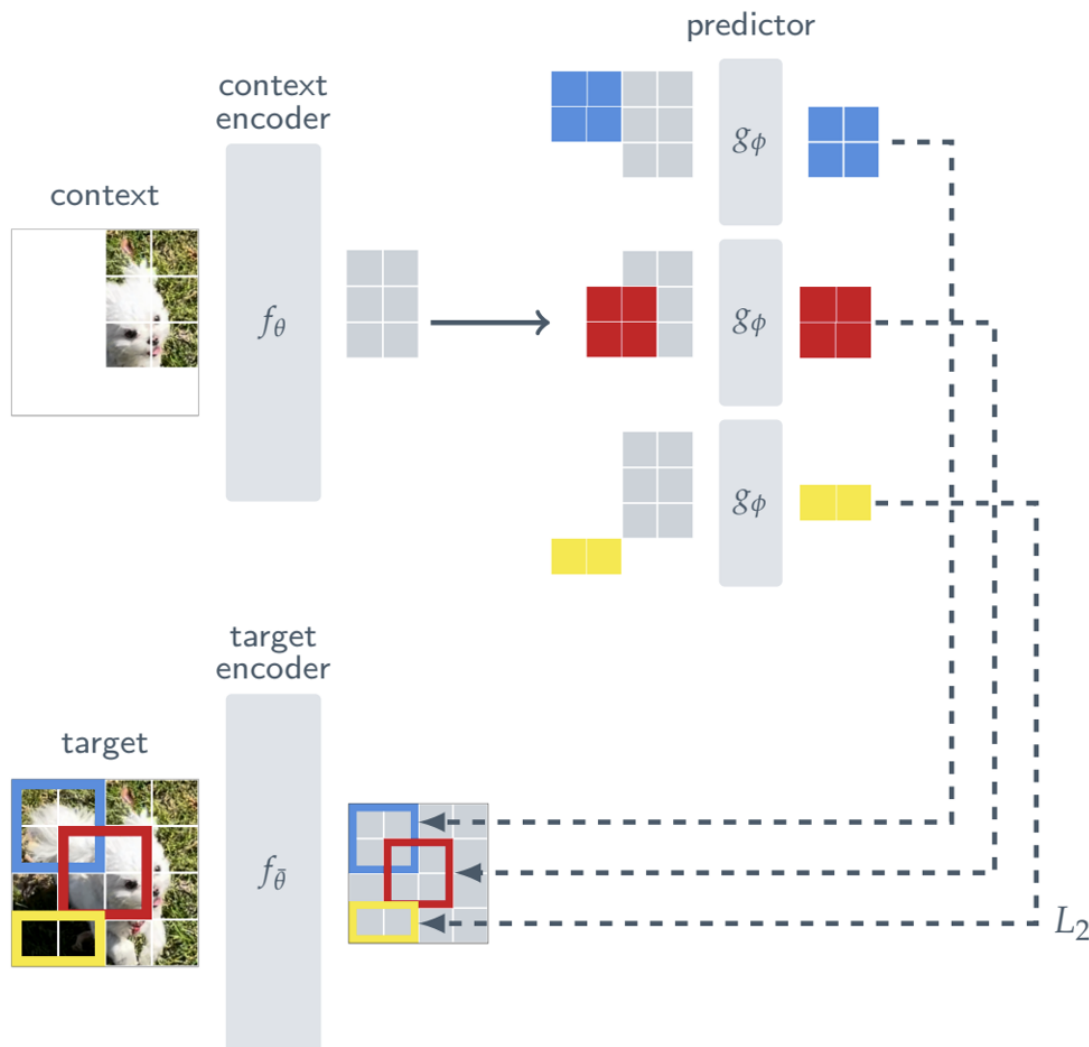
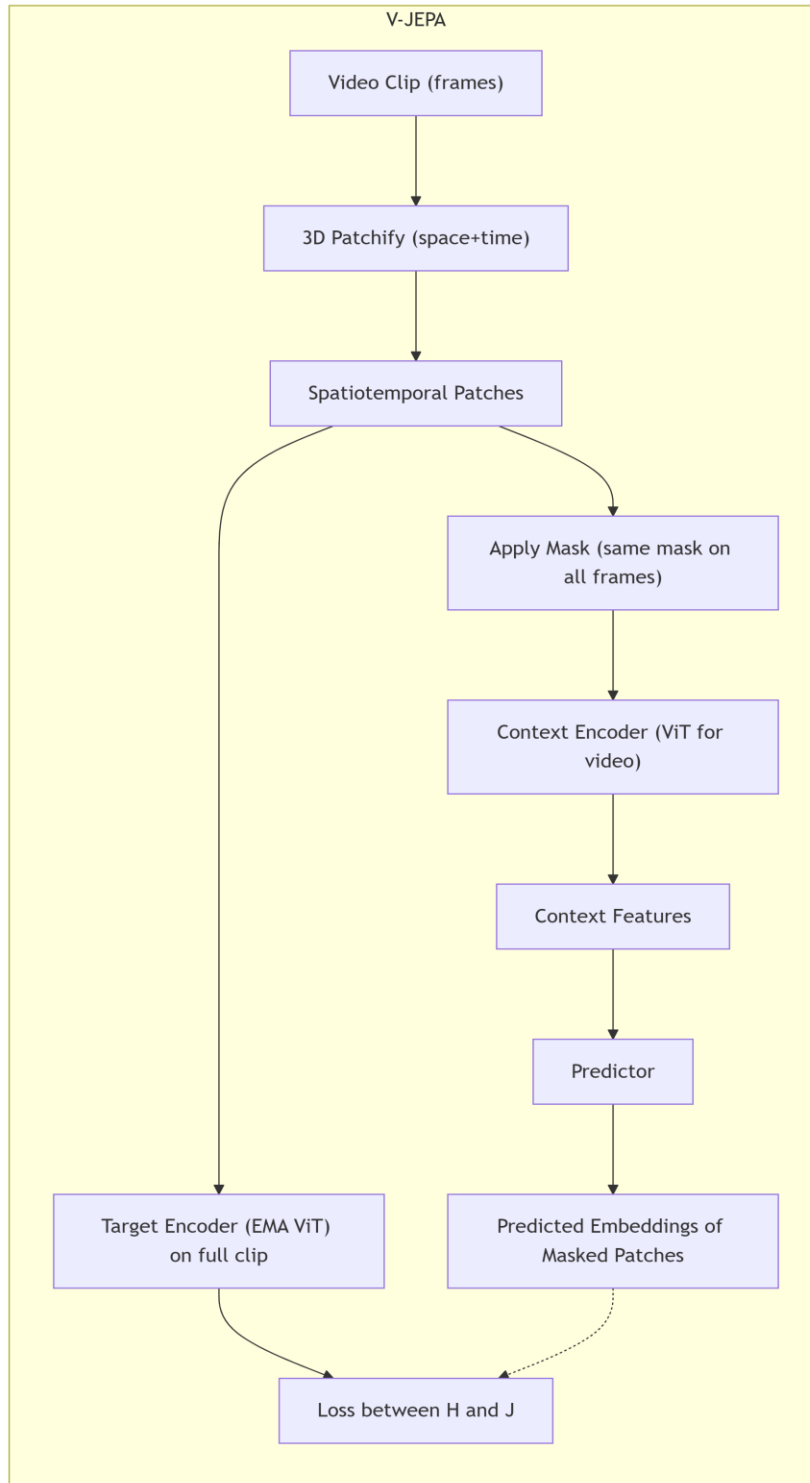


Figure: I-JEPA architecture for images (from [20]). The model masks out large blocks in one image and uses the remaining context to predict the representations of the masked regions. A target encoder (EMA) provides the true embeddings for the masked patches.

V-JEPA: JEPA for Video

V-JEPA generalizes JEPA to video by treating a video clip as a 3D tensor of frames[19][20]. A clip (e.g. 64 frames of $16 \times 224 \times 224$ pixels) is divided into 3D patches (e.g. $16 \times 16 \times 2$ sized blocks across space and two consecutive frames). Two masks are drawn: a *short-range* mask (covering small, random regions) and a *long-range* mask

(covering larger, sparse regions). Each mask is applied across all frames at once[21]. The context encoder then processes the video with masked patches removed, and the predictor is trained to reconstruct the embeddings of the masked patches using the unmasked ones[21][20].



Key point: unlike generative video models, V-JEPA **never reconstructs pixels** — it predicts in feature space[9][22]. This allows it to “discard unpredictable information” (like exact pixel changes) and focus on semantics. A V-JEPA model trained on millions of videos learns representations that excel at both motion-oriented and appearance-oriented tasks[22]. For example, a ViT-Huge trained solely with V-JEPA (no labels) achieved ~81.9% on Kinetics-400 and 72.2% on Something-Something-v2 with a *frozen* backbone[22]. In other words, V-JEPA produces *versatile* video features from unsupervised learning.

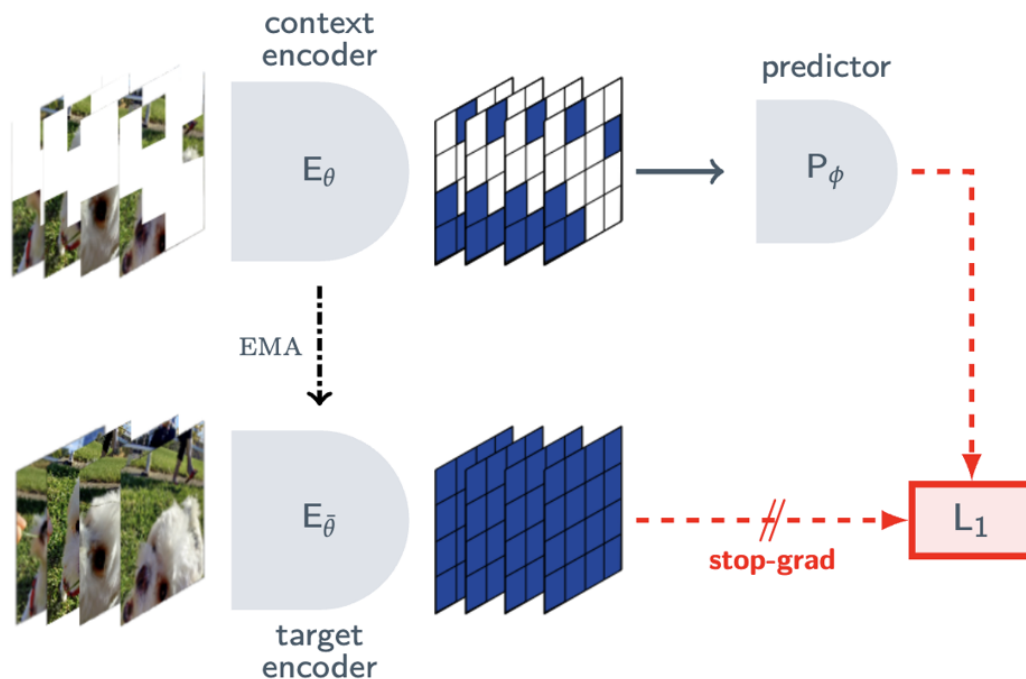
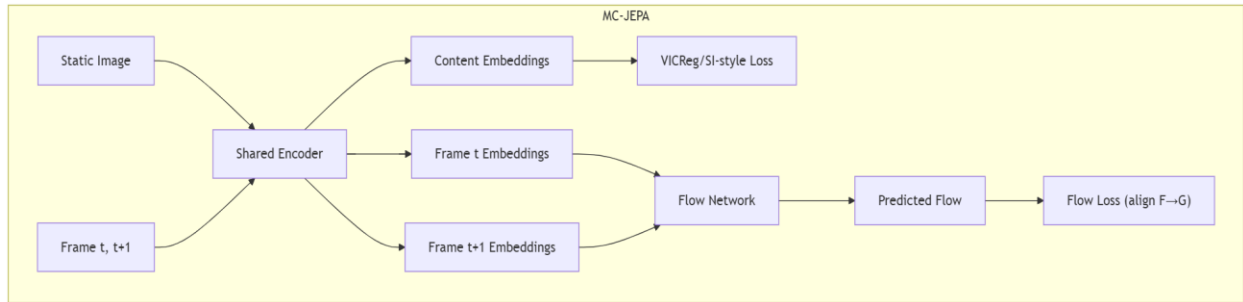


Figure: V-JEPA architecture (after [20]). The model splits a video clip into patches (space×time), masks out regions consistently across time, then uses the remaining context to predict the masked patches’ embeddings. A target encoder (EMA of the backbone) provides ground-truth features.

MC-JEPA: Motion and Content together

JEPA can also be extended to **multi-task learning**. MC-JEPA combines two objectives: optical flow estimation (learning motion) and standard content feature learning, sharing one encoder[23][24]. Concretely, it uses two tasks with a shared vision encoder: - *Content task*: standard SSL on static images (e.g. VICReg) to learn semantic content features.

- *Motion task*: self-supervised optical flow estimation on videos. Here, MC-JEPA takes two consecutive frames, uses the encoder to extract features, and trains a flow network (PWC-Net style) to predict how features move from frame to frame[23][25].



MC-JEPA's shared encoder thus learns features that encode both **what** is in the image and **how things move** between frames. This joint training yields state-of-the-art results on motion benchmarks (e.g. optical flow) *and* on vision tasks like segmentation[23][26]. In other words, the two JEPA-like objectives reinforce each other: motion cues improve content features and vice versa.

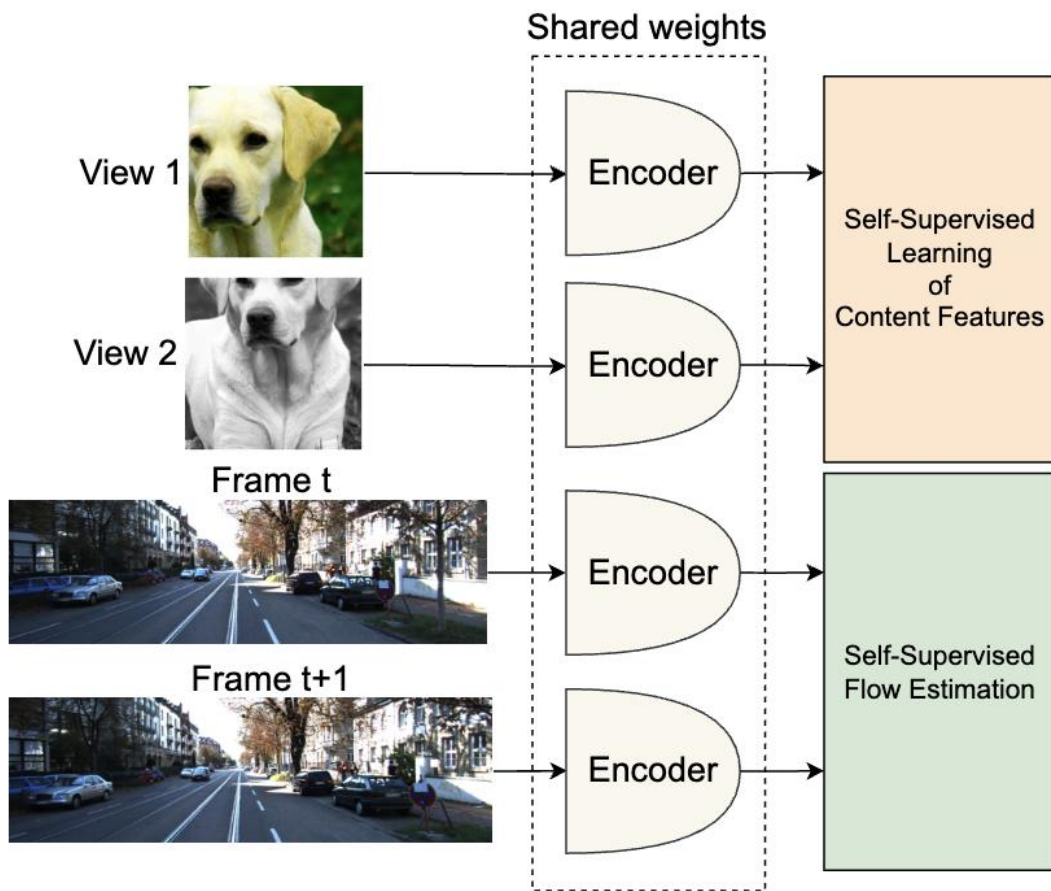
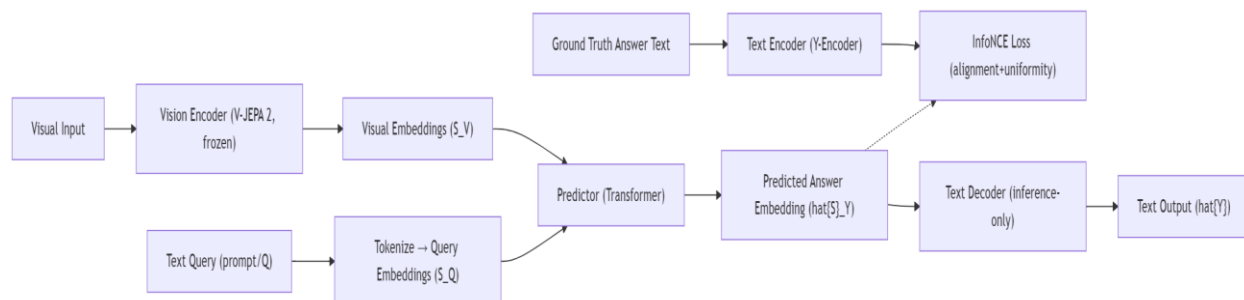


Figure: MC-JEPA architecture. A shared encoder processes both single images (for content SSL) and video frames (for flow estimation). The content branch (top) learns semantic features via standard SSL loss; the motion branch (bottom) learns to predict optical flow between frames. The joint training fuses motion and content in one JEPA framework[23].

VL-JEPA: JEPA for Vision–Language

VL-JEPA brings language into the picture by attaching a JEPA predictor to the video encoder. The **vision encoder** is a **frozen** V-JEPA 2 model (trained on unlabeled video) that outputs semantic visual embeddings[4][1]. The **predictor** is a smaller Transformer (8 layers of a pretrained Llama-3.2-1B) which takes the visual embeddings plus tokenized text query and outputs a *predicted text embedding* $\hat{S}_Y$ [4][3]. The **text encoder** (EmbeddingGemma) converts the ground-truth answer text into S_Y [4]. During training, VL-JEPA optimizes an InfoNCE loss between $\hat{S}_Y$ and S_Y , aligning the predicted and true semantics[3].

Key differences from standard VLMs: - **No autoregression**: The predictor attends bidirectionally to all vision and query tokens, and outputs $\hat{S}_Y$ in one pass[4]. There is no need to generate text token-by-token. - **Fewer parameters**: VL-JEPA uses ~1.6B total parameters (490M predictor + 300M encoder + 304M vision), roughly half the trainable size of comparable models[5]. - **Unified tasks**: The same model handles captioning, VQA, open-vocabulary classification, and retrieval without specialized heads. For example, to classify an image from a label set, VL-JEPA encodes each label into embeddings and picks the nearest one to $\hat{S}_Y$ [27].



The figure above summarizes VL-JEPA's components. At inference, the model can directly output $\hat{S}_Y$ and *only decode to text if needed* (e.g. for user display)[7].

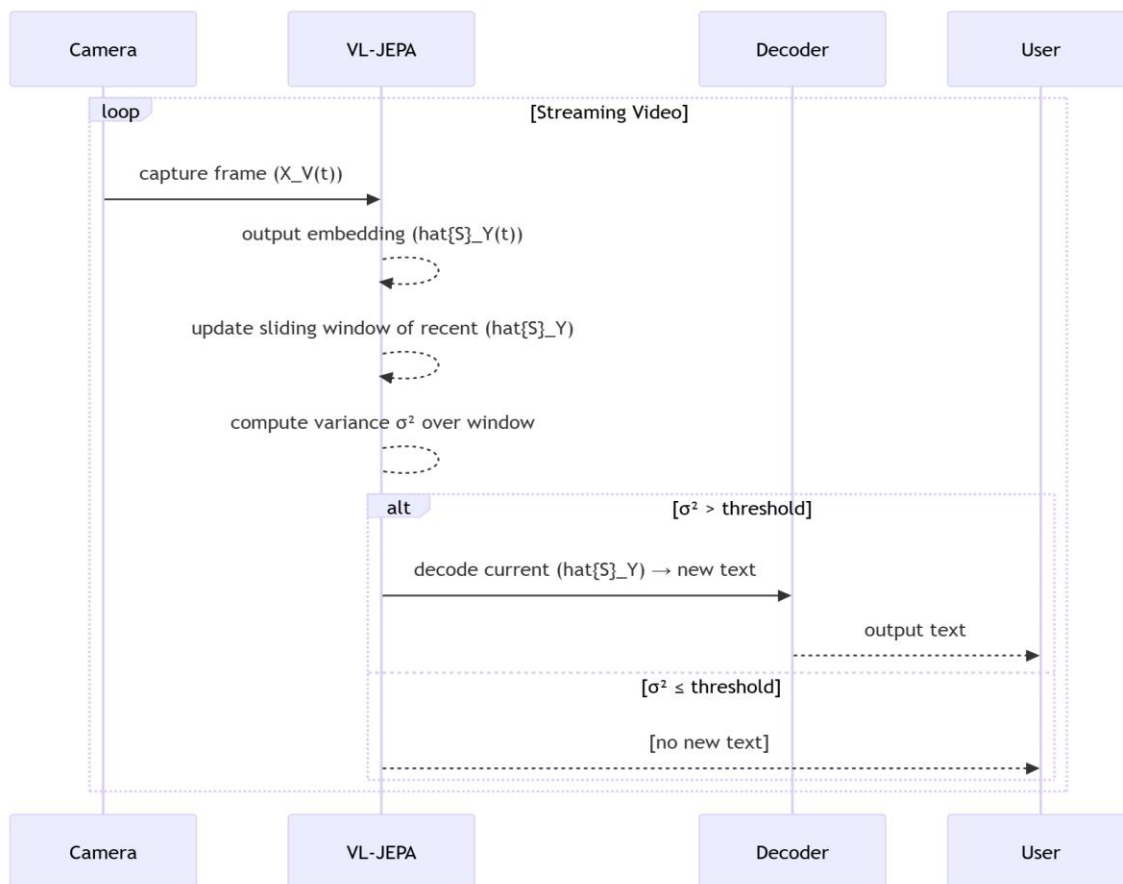
Training and Objectives

VL-JEPA is trained in two stages: large-scale pretraining (query-free) and supervised finetuning (with VQA/caption data). In pretraining, it uses billions of image/video+caption pairs to align vision and text in the joint embedding space. The **loss** is a bidirectional InfoNCE: it pulls each $\hat{S}_Y$ towards its true S_Y and simultaneously enforces that different samples' embeddings remain spread out (to avoid collapse)[3]. This is more effective than simple cosine or L2 loss; ablations show InfoNCE yields much better retrieval/classification performance[3].

Key hyperparameters: The embedding dimension is 1536. The target encoder (Y-Encoder) is updated with a very low learning-rate multiplier (0.05×) to keep embeddings stable early on[3]. The predictor is initialized from pretrained Llama layers to leverage prior language knowledge[4]. The vision encoder remains frozen, focusing training on cross-modal alignment.

Inference and Selective Decoding

A major advantage of JEPA is **real-time streaming inference**. Unlike autoregressive VLMs that must decode at fixed intervals, VL-JEPA produces a continuous *semantic stream* of embeddings $\hat{S}_Y(t)$ as video frames arrive. A simple algorithm can then detect when the meaning changes and only *decode* at those points. For example:



In words: as the video plays, VL-JEPA continuously outputs embeddings. Only when the embedding variance over a short window exceeds a threshold (signaling a new action or scene) do we invoke the text decoder. This **selective decoding** yields huge savings. In tests on long procedural videos, adaptive decoding at 0.35 Hz achieved the same captioning quality (CIDEr) as uniform 1 Hz decoding, cutting decode operations by $\sim 2.85\times$ [7][8]. In practical terms, for 1 hour of video this means $\sim 1,260$ decodes instead of 3,600. This efficiency is crucial for wearables or robots that must “see” continuously without the latency of token-by-token LLM generation.

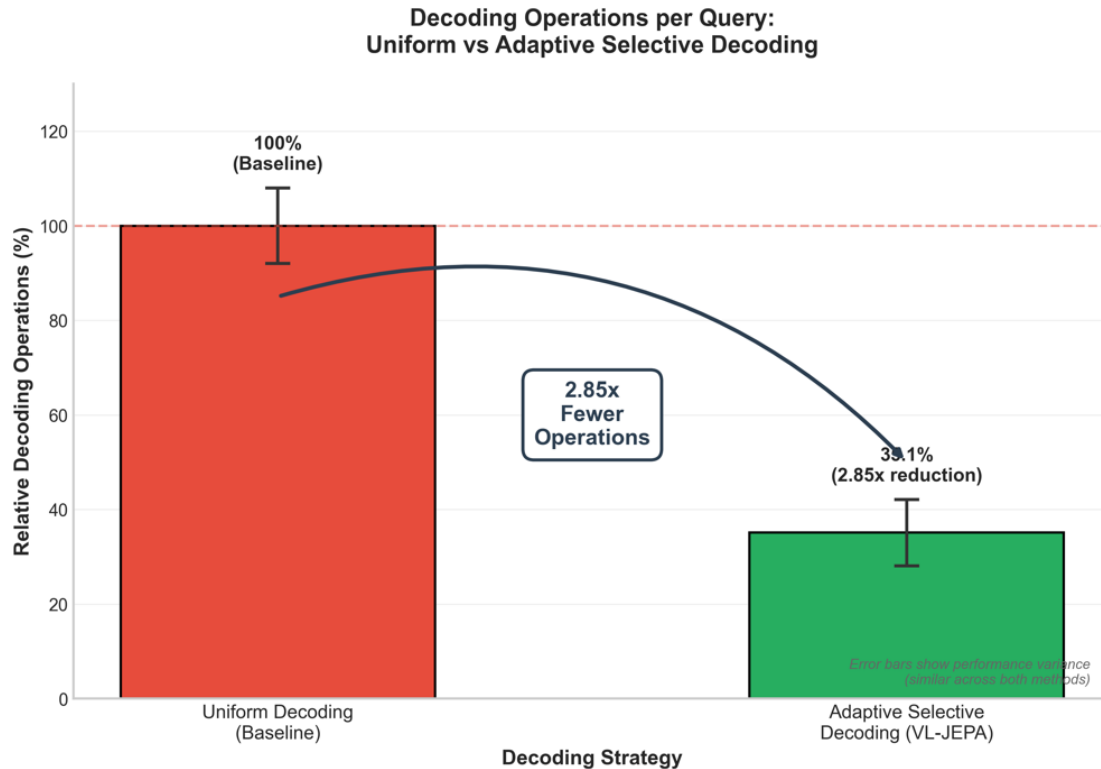


Figure: Decoding cost comparison. Uniform decoding (red) generates text at a fixed rate. VL-JEPA’s adaptive decoding (blue) monitors semantic change: when embeddings remain similar, no new decode is done. The result is 2.85× fewer decoding steps for the same captioning score[7][8].

Multi-Task Capabilities

Because VL-JEPA predicts embeddings, adding tasks is seamless. The *same* architecture handles multiple tasks:

- **Captioning and Open-Ended VQA:** Supply a prompt (e.g. “Describe this scene:” or “What is the person doing?”) as X_Q . The predictor outputs $\hat{S}_Y$, then the decoder yields a sentence. No extra heads needed.
- **Open-Vocabulary Classification:** Pre-compute embeddings S_{L_i} for each class label L_i . Given an image X_V , VL-JEPA predicts $\hat{S}_Y$ (with no question). The chosen class is the label whose embedding is closest to $\hat{S}_Y$ (by cosine similarity)[27].
- **Visual Question Answering (VQA):** Given multiple-choice answers A_j , encode each A_j into S_{A_j} . After predicting $\hat{S}_Y$, pick the answer whose embedding matches best. This makes VQA a *nearest-neighbor* in semantic space.
- **Text-to-Video Retrieval:** Encode a query X_Q into an embedding S_Q . For each video in the database, run the vision encoder + predictor (no decode) to get an embedding. Rank by cosine similarity with S_Q to retrieve matching videos.

All of the above use the same $\hat{S}_Y$ from the predictor; no task-specific fine-tuning or extra layers are needed. This unified design contrasts with traditional systems that require separate heads for classification or retrieval.

Performance: Surpassing Large Models

VL-JEPA's sample efficiency and embedding supervision translate to strong results. In controlled comparisons (same training data and encoder), embedding-prediction models achieve **2–3× higher accuracy** than token-based VLMs after the same training steps[11]. For example, after 5M video-caption samples, VL-JEPA was twice as good at captioning (CIDEr) and 8 points higher on video classification than a token-VLM baseline[11].

Benchmarks: The pretrained VL-JEPA_BASE (zero-shot) was evaluated on 8 video classification and 8 retrieval datasets. It **matched or exceeded** state-of-the-art contrastive or generative baselines using far less data: e.g. motion-centric tasks (Something-Something, EgoExo4D) saw +7% accuracy over best baseline[28]. On text-to-video retrieval, VL-JEPA_BASE tied the best model (58.4% average R@1 vs 58.1% for PE-Core) despite using 43× fewer training samples[28]. After finetuning on VQA data (VL-JEPA_SFT), it achieved competitive scores on GQA, TallyQA, and notably **best-in-class hallucination detection** (POPE)[28], all with just 1.6B parameters.

World Modeling / Causal Reasoning: Remarkably, VL-JEPA_SFT (1.6B) **outperformed GPT-4o, Claude 3.5 Sonnet, and Gemini-2.0 (~400B)** on the “inverse dynamics” task (WorldPrediction)[28]. This task asks which action caused a state change (e.g. what caused a door to go from closed to open). VL-JEPA achieved 65.7% vs ~55% for the giant LLMs. The reason is that VL-JEPA directly predicts the *physics* in latent space, rather than relying on a possibly lossy image caption for reasoning. In effect, its embedding-based predictions act like a world model more faithfully than natural-language reasoning[12][28].

In summary, **VL-JEPA delivers top-tier performance with far fewer resources**. It matches much larger multimodal LLMs on vision-language tasks, and even establishes a new state-of-the-art in causal vision reasoning, all by reframing the problem in semantic space[3][28].

Why This Matters

JEPA represents a paradigm shift away from “next-token” thinking. By predicting **meaning**, models focus on the *contents* of scenes and questions rather than linguistic minutiae. This makes learning more efficient (fewer parameters, faster training[5]) and inference more practical (low-latency streaming with selective output[7]). For real-world systems—smart glasses, robots, live video analytics—this efficiency is crucial.

At the same time, VL-JEPA retains the flexibility of generative models: it can *generate* any text output via its decoder, but does so more sparingly and only when needed. And it naturally handles open-ended tasks (generation, VQA) and closed tasks (classification, retrieval) in one framework[27]. In effect, the same continuous embeddings that speed up inference also enable multi-modal, multi-task versatility.

Limitations: Embedding-prediction is not a silver bullet for *all* AI problems. Tasks that require explicit step-by-step reasoning or planning (“chain-of-thought”) may still favor token-generative approaches. Similarly, language-intensive tasks that rely on factual knowledge (history, science) will likely need retrieval or prompting techniques outside pure JEPA. But for perception-driven tasks, JEPA’s performance and efficiency gains are compelling[11][3].

Summary

Joint Embedding Predictive Architectures (JEPA) reframe vision and vision–language AI by predicting latent representations instead of tokens. In I-JEPA and V-JEPA, models learn to predict masked image/video features; in VL-JEPA, a model predicts continuous answer embeddings. This approach **halves model size**, **doubles learning speed**, and **enables real-time selective decoding**, all while achieving state-of-the-art results on classification, retrieval, VQA, and even causal reasoning tasks[1][28]. By focusing on *meaning* first, then words later, JEPA models offer a practical and powerful path to the next generation of intelligent vision–language systems.

Sources: The above draws on recent Meta AI research on I-JEPA[15][18], V-JEPA[22], MC-JEPA[23], and VL-JEPA[1][28], as well as independent analyses[9][29]. All key claims are cited from these published sources.

[1] [3] [5] [7] [27] [28] VL-JEPA: Joint Embedding Predictive Architecture for Vision-language | OpenReview

<https://openreview.net/forum?id=tjimrhc2BU>

[2] [4] [6] [8] [9] [10] [11] [12] [13] [14] [20] [29] VL-JEPA: Why Predicting Embeddings Beats Generating Tokens for Vision-Language AI | rewire.it | rewire.it Blog

<https://rewire.it/blog/vl-jepa-why-predicting-embeddings-beats-generating-tokens/>

[15] [18] [2301.08243] Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture

<https://arxiv.org/abs/2301.08243>

[16] [17] [19] [21] [24] [25] Deep Dive into Yann LeCun’s JEPA | Rohit Bandaru

<https://rohitbandaru.github.io/blog/JEPA-Deep-Dive/>

[22] [2404.08471] Revisiting Feature Prediction for Learning Visual Representations from Video

<https://arxiv.org/abs/2404.08471>

[23] [26] [2307.12698] MC-JEPA: A Joint-Embedding Predictive Architecture for Self-Supervised Learning of Motion and Content Features

<https://ar5iv.labs.arxiv.org/html/2307.12698>